

dirac_wavepacket: wave-packet transport in two-dimensional tilted Dirac and Weyl systems

Can Yesilyurt^{a,*}

^a*Nanoelectronics Research Center, Istanbul, Turkey*

Abstract

dirac_wavepacket is an open-source Python package for time-dependent transport of wave packets in two-dimensional Dirac and Weyl systems with anisotropic Fermi velocities and tilted cones. A two-component spinor is evolved on a uniform grid by a second-order symmetric split-operator Fourier scheme whose momentum step is an analytical 2×2 unitary. Arbitrary polygonal or rotated barrier geometries, absorbing drain contacts that double as transmission and reflection detectors, mass-confined reflecting walls, and an optional 4-component coupled propagator for intervalley coupling are supported together in a single workflow. The code is installed via **pip**, validated by a pytest suite, and released under the MIT licence.

Keywords: Dirac equation, Weyl semimetal, tilted cone, valley transport, split-operator FFT, wave-packet dynamics

Required Metadata

Current code version

1. Motivation and significance

Two-dimensional Dirac and Weyl semimetals with tilted, anisotropic low-energy dispersions have become a central platform for valley-resolved electron optics and for all-electrostatic proposals of valley filtering, Veselago focusing, and phase gating [1, 2, 3, 4]. Materials of current interest include 8-Pmmn

*Corresponding author.

Email address: `cyesil@nanorescenter.com` (Can Yesilyurt)

Nr.	Code metadata description	Value
C1	Current code version	v1.0.0
C2	Permanent GitHub link to code/repository used for this code version	github.com/can-yesilyurt/Dirac-Wavepacket
C3	Legal Code License	MIT
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	Python (NumPy, SciPy, Matplotlib, PyYAML); optional pyFFTW for FFT acceleration; pytest for tests
C6	Compilation requirements, operating environments & dependencies	Python ≥ 3.11 . Platform-independent. <code>pip install -e .</code> or <code>pip install -e ".[fft]"</code> for the pyFFTW extra.
C7	If available, link to developer documentation/manual	github.com/can-yesilyurt/Dirac-Wavepacket/tree/main/docs
C8	Support email for questions	cyesil@nanorescenter.com

Table 1: Code metadata (mandatory).

borophene [5, 6], α -(BEDT-TTF) $_2$ I $_3$ [2], the transition-metal tellurides WTe $_2$ and MoTe $_2$ [7, 8], and strained-graphene variants. In these systems the tilt term acts on the pseudospin identity and rigidly displaces the Fermi contour along the tilt direction; at the two inequivalent valleys K and K' , crystal symmetry enforces opposite tilt, which breaks the angular symmetry of transmission across electrostatic interfaces and enables electrostatic valley control without external magnetic fields or strain.

Quantitative device design in this regime requires solving the time-dependent 2D Dirac equation for a two-component spinor in a spatially inhomogeneous electrostatic landscape, with realistic boundary conditions: absorbing source and drain contacts, reflecting channel walls that respect Klein tunneling, and barriers of arbitrary shape or rotation. Several existing open-source tools touch neighbouring regions of this problem space. KWANT [11] and the time-dependent extension TKWANT [12] provide tight-binding scattering-matrix calculations but are lattice-based and stationary in their core use. DIRAC++ [13] is a 2+1D Dirac split-operator code for atomic physics, not for condensed-matter transport. PYTALISES [14] and QMSOLVE [15] solve the time-dependent Schrödinger equation with scalar wavefunctions and do not treat the Dirac Hamiltonian, pseudospin-momentum locking, or valley chirality. To our knowledge, no publicly available package combines continuum-Dirac time-dependent transport, anisotropic and tilted dispersions, arbitrary

barrier geometries, drain contacts as transmission / reflection detectors, and optional intervalley coupling in a single installable workflow.

`dirac_wavepacket` fills this gap. It is a pure-Python package built around a second-order Strang-split Fourier propagator with an analytical 2×2 momentum-space unitary. It is used in our group for valley-polarised transport studies [16, 17] and is being released for general use in device-level electron-optics modelling. A typical user defines a YAML configuration describing the grid, dispersion, injection, potential, and boundaries; invokes `run_simulation()` either directly or through one of the parallel sweep drivers; and analyses the resulting transport observables and snapshot series.

2. Software description

2.1. Software architecture

`dirac_wavepacket` is organised around the computational pipeline. A `SimConfig` dataclass aggregates nested configurations for the grid, physics, wavepacket, potential, absorber, time integration, detector, and coupling. Five subsystems cooperate at runtime (Fig. 1): a grid builder constructs the real- and momentum-space arrays; a wavepacket initialiser places a Gaussian envelope with the correct valley eigenspinor; a potential builder assembles rectangular, p-n junction, shaped parallelogram, polygonal, or additive multi-barrier landscapes; one of two propagators (2-spinor default or 4-spinor coupled) evolves the wavefunction; and an absorber / detector pair records $T(t)$, $R(t)$, $P_y(t)$ and the valley polarisation $\eta(t)$.

The split-operator Fourier scheme advances $\psi(t + \Delta t) = e^{-iH\Delta t}\psi(t)$ through $e^{-iH_R\Delta t/2} e^{-iH_K\Delta t} e^{-iH_R\Delta t/2}$, where H_R is diagonal in position space and H_K in momentum space. The momentum step is an analytical 2×2 unitary per $\mathbf{k}$ -point that incorporates the tilt, anisotropic Fermi velocities, and valley chirality directly; no eigenvector storage is needed. Reflecting walls are implemented by a local mass profile $M(y)\sigma_z$, which opens a gap at the boundary and prevents Klein-tunneling leakage that a scalar wall would permit [9]. Absorbing drains use a $\cos^2$ profile; the absorbed probability is *accumulated* into cumulative $T(t)$ and $R(t)$ counters so that the probability budget $T + R + P_y + P_{\text{rem}} = 1$ closes at every step by construction.

2.2. Software functionalities

The public API is a small set of top-level imports: `SimConfig`, `load_config`, `run_simulation`, plus the grid, wavepacket, potential, and detector builders. A typical YAML configuration is loaded and run in three lines:

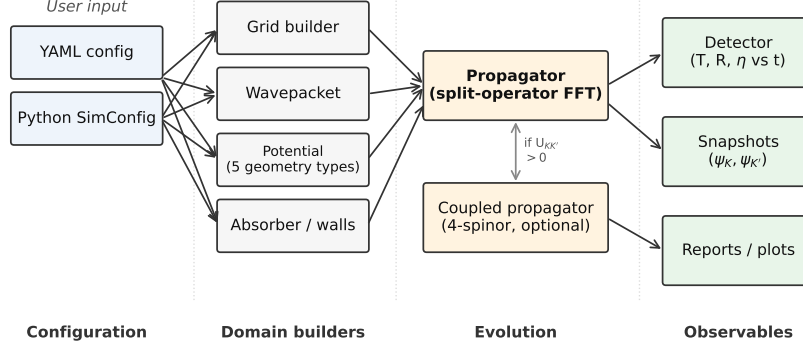


Figure 1: Architecture of `dirac_wavepacket`. Inputs (YAML or Python dataclass) flow through grid, wavepacket, and potential builders into the propagator; the absorber captures transmitted and reflected probability into the detector, which records the full transport record. An optional 4-spinor coupled propagator is selected automatically when intervalley coupling is enabled.

```
from dirac_wavepacket import load_config, run_simulation
cfg = load_config("my_config.yaml")
result = run_simulation(cfg, save_wf=True)
```

Four features distinguish the package from general-purpose Schrödinger solvers.

Tilted and anisotropic dispersions. The physics configuration accepts independent Fermi velocities v_x, v_y and a tilt vector (w_x, w_y) that is reversed automatically at the K' valley by the propagator. Dual-valley runs launch both valleys in parallel and report the time-resolved valley polarisation $\eta(t) = (T_K - T_{K'}) / (T_K + T_{K'})$.

Arbitrary barrier geometry. The potential builder supports rectangular, p–n junction, rotated parallelogram, arbitrary (convex or non-convex) polygonal, and additive multi-barrier landscapes, with optional sigmoid interface smoothing. Rotated barriers of any angle are specified by four edge positions and are the basis of the all-electrostatic valley filter in our first example.

Drain contacts as detectors. Absorbing drains and channel walls are implemented consistently with the Dirac structure and their absorption is bookkept as cumulative $T(t), R(t), P_y(t)$ histories. Auto-stop on small residual probability is enabled by default and typically shortens runs by 2–3 $\times$.

Optional intervalley coupling. A 4-spinor coupled propagator supports a spatially local, pseudospin-preserving coupling $U_{KK'}(x, y) \cdot \mathbb{I}_2$; the propaga-

tor automatically reduces to two independent 2-spinor evolutions when the coupling amplitude is zero. This equivalence is explicitly asserted by the test suite.

Three console-script sweep drivers (`dwp-sweep`, `dwp-sweep-v0`, `dwp-sweep-vsd`) wrap the simulation for V_0 , geometry, and source–drain bias scans with per-task JSON checkpoints and `--resume` support.

3. Illustrative examples

Two self-contained example scripts in the `examples/` directory exercise the core capabilities. Both support a `--quick` flag for laptop-scale smoke testing and a publication-quality default mode. Full input parameters are in the scripts; here we summarise inputs and outputs only.

3.1. Example 1: all-electrostatic valley filter

A Gaussian wave packet is launched at normal incidence through a rectangular barrier ($V_0 = 1.75 E_F$) in a channel with transverse tilt $w_y = 0.3 v_F$. Two runs are compared via a `--barrier` flag (Fig. 2): a straight control barrier at $\alpha = 0$ and a rotated filtering barrier at $\alpha = 20^\circ$. The results (Table 2) are $T_K = T_{K'} = 0.976$, $\eta = 0$ for the control and $T_K = 0.716$, $T_{K'} = 0.416$, $\eta = +0.265$ for the filter. The tilt alone does not produce polarisation at a straight barrier; rotating the same barrier by 20° is sufficient to generate a large valley polarisation by purely electrostatic means.

Barrier	T_K	$T_{K'}$	η
rect ($\alpha = 0$)	0.976	0.976	0.000
angled ($\alpha = 20^\circ$)	0.716	0.416	+0.265

Table 2: Example 1 transmission results; $\eta = (T_K - T_{K'})/(T_K + T_{K'})$.

3.2. Example 2: angular dependence of Klein tunneling

The second example exercises the untilted, isotropic Dirac limit. A rectangular barrier at $V_0 = 2 E_F$ is probed by a single-valley wave packet at a list of injection angles; at zero tilt K and K' are equivalent. Four angles $\{-15^\circ, -5^\circ, +5^\circ, +15^\circ\}$ are scanned in a single invocation (Fig. 3). The transmission peaks near $\theta = 0$ (the Klein paradox), and drops off with $|\theta|$; $T(\pm 5^\circ) = 0.974$, $T(\pm 15^\circ) = 0.868$. The mirror-symmetry residuals between

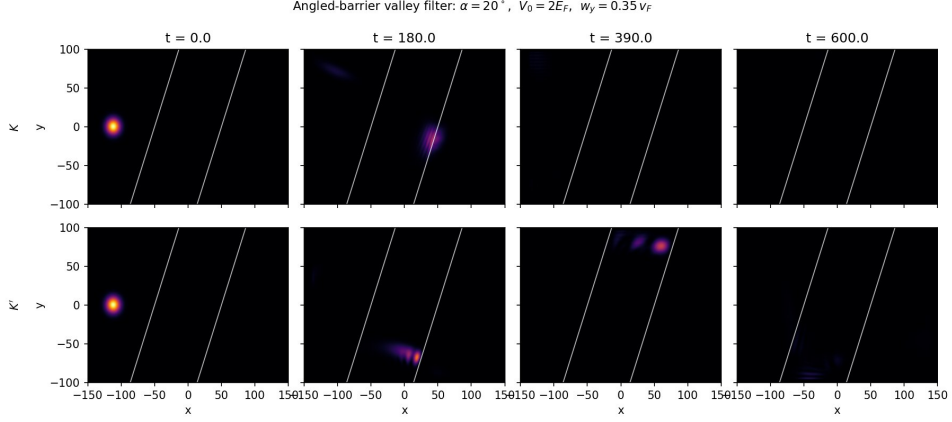


Figure 2: Example 1 (angled barrier, $\alpha = 20^\circ$). Top row: K wave packet; bottom row: K' . Barrier outlines in blue, drain regions shaded. K transmits cleanly to the right drain while K' is reflected back, yielding $\eta = +0.265$.

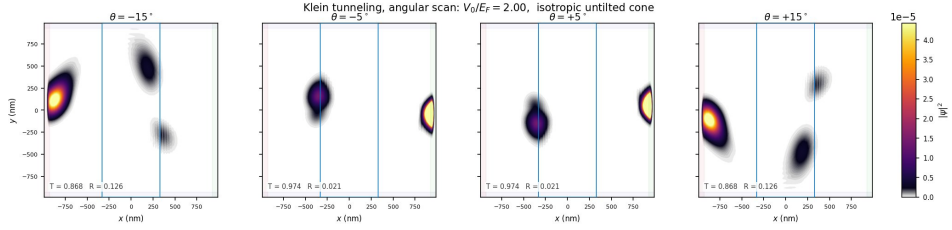


Figure 3: Example 2 Klein-tunnelling angular scan. Four single-valley runs at $\theta \in \{-15^\circ, -5^\circ, +5^\circ, +15^\circ\}$ through a rectangular barrier of height $V_0 = 2 E_F$. Mirror-symmetric pairs ($\theta \leftrightarrow -\theta$) are numerically coincident to $|\Delta T| \sim 10^{-6}$.

$\pm\theta$ pairs are $|\Delta T| \sim 10^{-6}$, confirming the propagator's numerical symmetry at single precision.

A pytest suite (eight tests, ~ 40 s total) asserts norm conservation, mirror symmetry, coupled-independent equivalence at zero intervalley coupling, polygon / shaped barrier equivalence, and Klein-paradox agreement with the analytical Katsnelson transfer-matrix formula [10].

4. Impact

`dirac_wavepacket` enables new research questions at the intersection of Dirac / Weyl electron optics and realistic device modelling. Our group has used it as the computational engine for studies of valley-resolved transport in tilted Dirac channels [16] and for an analysis of all-electrostatic valley phase

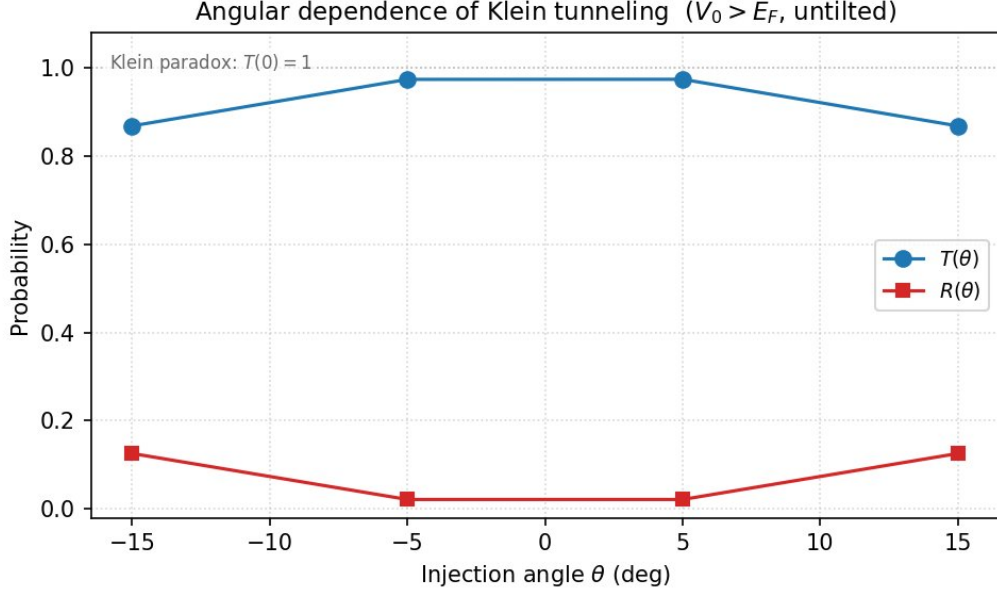


Figure 4: Summary of Example 2: $T(\theta)$ (blue) approaches the Klein-paradox limit $T = 1$ near $\theta = 0$ and decreases with $|\theta|$; $R(\theta)$ (red) is the complement.

gates [17]; both manuscripts cite the package as their numerical tool. The package enables systematic parameter scans (barrier geometry, tilt magnitude and direction, intervalley coupling amplitude, source–drain bias) that are impractical with tight-binding scattering-matrix codes on realistic device sizes, and it resolves time-dependent information (transient $\eta(t)$, wave-packet trajectories, Fabry–Pérot build-up) that is inaccessible to stationary methods.

Beyond our own use, `dirac.wavepacket` is intended as an installable tool for the broader 2D-materials device community. The installation is `pip install dirac-wavepacket` from the archived source; no compilation is required, no GPU is mandatory, and a typical production run completes in 10–30 minutes on a standard workstation. The YAML configuration interface means users can reproduce published simulations by sharing a config file rather than a full code diff. The pytest suite and auto-grid validator give new users a fast way to check that a chosen grid / time step combination is numerically sound before launching long runs. Compared with the continuum-Schrödinger neighbours PYTALISES [14] and QMSOLVE [15] the package occupies a distinct physical regime: the Dirac structure, anisotropic / tilted dispersion, valley chirality, and Klein-regime-compatible boundaries are not available in the Schrödinger family of solvers.

5. Conclusions

We have presented `dirac_wavepacket`, a compact and installable Python package for time-dependent transport of wave packets in 2D Dirac and Weyl systems with tilted, anisotropic cones. The core is a second-order split-operator Fourier propagator with an analytical 2×2 momentum-space unitary and a validated optional 4-component coupled extension. Arbitrary polygonal or rotated barriers, drain-as-detector bookkeeping, and mass-wall confinement are supported out of the box. Two worked examples demonstrate the key capabilities and the test suite locks them down with tight physical invariants. Planned extensions include GPU acceleration via CuPy and a companion package for valley-resolved phase extraction from saved wavefunction snapshots.

Acknowledgements

The author thanks A.D. Zabolotskii (Institute for Spectroscopy RAS) for correspondence on the 8-Pmmn borophene tight-binding model and for clarifying which borophene phases have been experimentally synthesised, and the Nanoelectronics Research Center for computing resources.

References

- [1] A.H. Castro Neto, F. Guinea, N.M.R. Peres, K.S. Novoselov, A.K. Geim, The electronic properties of graphene, *Rev. Mod. Phys.* 81 (2009) 109–162.
- [2] M.O. Goerbig, J.-N. Fuchs, G. Montambaux, F. Piéchon, Tilted anisotropic Dirac cones in quinoid-type graphene and α -(BEDT-TTF)₂I₃, *Phys. Rev. B* 78 (2008) 045415.
- [3] V.H. Nguyen, J.-C. Charlier, Klein tunneling and electron optics in Dirac–Weyl fermion systems with tilted energy dispersion, *Phys. Rev. B* 97 (2018) 235113.
- [4] S.-H. Zhang, W. Yang, Oblique Klein tunneling in 8-Pmmn borophene p–n junctions, *Phys. Rev. B* 97 (2018) 235440.
- [5] A. Lopez-Bezanilla, P.B. Littlewood, Electronic properties of 8-Pmmn borophene, *Phys. Rev. B* 93 (2016) 241405(R).

- [6] A. D. Zabolotskiy, Yu. E. Lozovik, Strain-induced pseudomagnetic field in the Dirac semimetal borophene, *Phys. Rev. B* 94 (2016) 165403.
- [7] A. A. Soluyanov et al., Type-II Weyl semimetals, *Nature* 527 (2015) 495–498.
- [8] Q. Wang, C. Yesilyurt et al., Room-temperature nonlinear Hall effect and wireless radio-frequency rectification in Weyl semimetal T_d -MoTe₂, *Nano Lett.* 19 (2019) 2647–2653.
- [9] M. V. Berry, R. J. Mondragon, Neutrino billiards: Time-reversal symmetry-breaking without magnetic fields, *Proc. R. Soc. Lond. A* 412 (1987) 53–74.
- [10] M. I. Katsnelson, K. S. Novoselov, A. K. Geim, Chiral tunnelling and the Klein paradox in graphene, *Nat. Phys.* 2 (2006) 620–625.
- [11] C. W. Groth, M. Wimmer, A. R. Akhmerov, X. Waintal, Kwant: a software package for quantum transport, *New J. Phys.* 16 (2014) 063065.
- [12] T. Kloss, J. Weston, B. Gaury, B. Rossignol, C. Groth, X. Waintal, Tk-want: a software package for time-dependent quantum transport, *New J. Phys.* 23 (2021) 023025.
- [13] G. R. Mocken, C. H. Keitel, FFT-split-operator code for solving the Dirac equation in 2+1 dimensions, *Comput. Phys. Commun.* 178 (2008) 868–882.
- [14] S. Vowe, pytalises: an easy-to-use Python implementation of the split-step Fourier method for the time-dependent Schrödinger equation (2020). <https://github.com/savowe/pytalises>.
- [15] quantum-visualizations, qmsolve: a module for solving and visualising the Schrödinger equation (2022). <https://github.com/quantum-visualizations/qmsolve>.
- [16] C. Yesilyurt, All-electrostatic valley filtering by barrier rotation in tilted Dirac channels, manuscript submitted (2026).
- [17] C. Yesilyurt, All-electrostatic valley phase gates in tilted Dirac channels, manuscript in preparation (2026).

Current executable software version

Nr.	(Executable) software metadata description	Value
S1	Current software version	v1.0.0
S2	Permanent link to executables of this version	github.com/can-yesilyurt/Dirac-Wavepacket/releases/tag/v1.0.0
S3	Legal Software License	MIT
S4	Computing platforms / Operating Systems	Linux, macOS, Windows (any platform supporting CPython ≥ 3.11).
S5	Installation requirements & dependencies	Python ≥ 3.11 ; NumPy, SciPy, Matplotlib, PyYAML; optional pyFFTW. Install via <code>pip install -e .</code> from the cloned repository.
S6	If available, link to user manual	github.com/can-yesilyurt/Dirac-Wavepacket/tree/main/docs
S7	Support email for questions	cyesil@nanorescenter.com

Table 3: Executable software metadata (mandatory).